

Índice

INTRODUCCIÓN	1
Propósito de la guía	1
CAPÍTULO I. PLANTEAMIENTO GENERAL DEL TEMA	1
1.1. Descripción del tema	1
1.2. Importancia del uso de librerías en Python	2
1.3. Área de aplicación seleccionada	2
CAPÍTULO II. OBJETIVOS DEL TRABAJO	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
2.3. Coherencia de objetivos	3
CAPÍTULO III. MARCO TEÓRICO	3
3.1. Concepto de Python	3
3.2. Concepto de librería en programación	3
3.3. Clasificación de librerías de Python	3
3.4. Importancia de las librerías en problemas reales	4
CAPÍTULO IV. DESARROLLO DE LAS LIBRERÍAS SELECCIONADAS	4
4.1. LIBRERÍA N.º 1: hashlib	4
4.2. LIBRERÍA N.º 2: yara-python	6
4.3. LIBRERÍA N.º 3: impacket	8
CAPÍTULO V. CUADRO COMPARATIVO DE LAS LIBRERÍAS	10
Análisis interpretativo del cuadro comparativo	11
CAPÍTULO VI. PARTE PRÁCTICA: ANÁLISIS FORENSE DE APKs	11
6.1. Plataforma utilizada	11
6.2. Descripción del problema práctico	11
6.3. Datos utilizados	12
6.4. Código desarrollado	12
6.5. Explicación del código	13
6.6. Resultados obtenidos	13
6.7. Evidencias de ejecución	13
CAPÍTULO VII. CASO PRÁCTICO: DETECCIÓN DE APK ENVENENADO	14
7.1. Descripción del caso	14
7.2. Problema identificado	14
7.3. Solución propuesta con Python	14
7.4. Beneficios de la solución	14
CONCLUSIONES	15
RECOMENDACIONES	15
REFERENCIAS BIBLIOGRÁFICAS	16
ANEXOS	16
Anexo 1: Código fuente completo	16
Anexo 2: Resultados de ejecución	16
Anexo 3: Archivos de prueba utilizados	17
Anexo 4: Enlaces a notebooks de Google Colab	17
Anexo 5: Repositorio del proyecto	17
Anexo 6: Regla YARA utilizada	17

INTRODUCCIÓN

Python se ha consolidado como uno de los lenguajes de programación más relevantes en el campo de la seguridad informática. Su sintaxis clara, la disponibilidad de miles de librerías

especializadas y su extensa comunidad lo convierten en la herramienta de elección para analistas, investigadores y profesionales de la ciberseguridad defensiva en todo el mundo.

El presente trabajo de investigación monográfico examina tres librerías fundamentales del ecosistema Python orientadas a la ciberseguridad: yara-python, hashlib e impacket. Estas herramientas permiten, respectivamente, detectar y clasificar malware mediante reglas de reconocimiento de patrones, verificar la integridad de archivos y mensajes mediante funciones hash criptográficas, y analizar protocolos de red como SMB, Kerberos y LDAP en entornos de laboratorio.

Como componente práctico principal, se desarrolló un caso real de análisis forense sobre dos aplicaciones Android (APK): una original (CairosOriginal.apk) y otra infectada con un payload de Metasploit (CairosVenom.apk). Se utilizó hashlib para calcular y comparar los hashes criptográficos de ambos archivos, evidenciando el efecto avalancha, y se implementó un detector basado en yara-python y Androguard capaz de identificar automáticamente el APK envenenado mediante el análisis de permisos, subpaquetes inyectados y firmas DEX. Todo el código fuente, los APK de prueba y este documento se encuentran disponibles en el repositorio del proyecto.

El informe se organiza siguiendo el esquema de investigación institucional: inicia con el planteamiento del problema y los objetivos, continúa con el marco teórico y el desarrollo detallado de cada librería, incluye una práctica ejecutada en Google Colab, describe un caso práctico en contexto real y concluye con el análisis comparativo y las conclusiones del grupo.

Este trabajo busca que los estudiantes no solo conozcan la definición de las librerías, sino que comprendan su utilidad práctica dentro de operaciones de ciberseguridad defensiva, siempre respetando marcos éticos y legales vigentes.

Propósito de la guía

- Orientar la elaboración del trabajo monográfico sobre librerías de Python aplicadas a la ciberseguridad.
- Relacionar la investigación teórica con una práctica funcional ejecutada en notebook.
- Promover el análisis de casos reales y la interpretación de resultados en contextos de seguridad informática.
- Demostrar el valor defensivo de yara-python, hashlib e impacket en entornos académicos y profesionales.

CAPÍTULO I. PLANTEAMIENTO GENERAL DEL TEMA

1.1. Descripción del tema

Las librerías de Python son componentes de software precompilados que ofrecen funcionalidades específicas listas para ser integradas en proyectos de programación. En el ámbito de la ciberseguridad, estas librerías reducen drásticamente el tiempo necesario para implementar mecanismos defensivos, ya que los analistas pueden enfocarse en la lógica de detección y respuesta en lugar de construir desde cero cada función criptográfica o de análisis de protocolos.

El problema que este trabajo aborda es la necesidad de contar con herramientas de detección de amenazas, verificación de integridad y análisis de tráfico de red que sean accesibles, documentadas y aplicables en entornos educativos. Las tres librerías seleccionadas — yara-python, hashlib e impacket — responden directamente a esta necesidad: yara-python permite identificar malware mediante firmas de patrones; hashlib garantiza la integridad de archivos mediante funciones hash como SHA-256 y MD5; e impacket facilita el análisis y la simulación de protocolos de red como SMB y Kerberos en laboratorio.

1.2. Importancia del uso de librerías en Python

Las librerías permiten reutilizar código ampliamente probado por la comunidad, reducir errores humanos en implementaciones críticas de seguridad y acceder a algoritmos complejos con interfaces de programación simples. En ciberseguridad, donde un error de implementación puede comprometer la totalidad de un sistema, el uso de librerías estandarizadas resulta indispensable.

Además, las librerías facilitan el trabajo colaborativo entre equipos de seguridad, la integración con plataformas de análisis de amenazas y la automatización de tareas repetitivas como el cálculo masivo de hashes o el escaneo de archivos con reglas YARA.

1.3. Área de aplicación seleccionada

Este trabajo se ubica en el área de ciberseguridad defensiva. Las librerías seleccionadas contribuyen de forma complementaria a la protección de sistemas: hashlib para integridad, yara-python para detección de código malicioso e impacket para análisis de protocolos de red en entornos controlados.

Pregunta guía	Respuesta desarrollada
¿Qué área eligieron?	Ciberseguridad defensiva: yara-python, hashlib e impacket.
¿Por qué eligieron esa área?	La ciberseguridad es una competencia crítica en la formación tecnológica actual. Las amenazas digitales crecen constantemente y las organizaciones requieren profesionales capacitados en herramientas defensivas.
¿Qué problema real puede resolver?	Detección de malware en archivos descargados, verificación de integridad de software distribuido en red, y análisis de tráfico SMB/Kerberos en auditorías internas de seguridad.
¿Qué resultados esperan obtener?	Generación de hashes verificables, detección de patrones maliciosos en archivos de prueba, y captura de paquetes de protocolos de red en laboratorio.

CAPÍTULO II. OBJETIVOS DEL TRABAJO

2.1. Objetivo general

Analizar el uso, características y aplicación práctica de las librerías yara-python, hashlib e impacket de Python orientadas a la ciberseguridad defensiva, mediante una investigación monográfica y el desarrollo de un notebook funcional en Google Colab.

2.2. Objetivos específicos

1. Identificar las principales características, funciones y métodos de las librerías yara-python, hashlib e impacket.
2. Explicar el funcionamiento básico, la instalación y la importación de cada librería, junto con sus métodos principales.
3. Comparar las tres librerías según su uso principal, nivel de dificultad, ventajas, limitaciones y campo de aplicación real.
4. Desarrollar una práctica en Google Colab utilizando las tres librerías para resolver un problema de ciberseguridad defensiva.
5. Interpretar los resultados obtenidos y relacionarlos con un caso práctico en contexto de seguridad informática institucional.

2.3. Coherencia de objetivos

Elemento	Verificación
Título	Menciona librerías de Python y el área de ciberseguridad.
Objetivo general	Coincide con la finalidad del trabajo: análisis e implementación práctica.
Objetivos específicos	Organizan el desarrollo del informe de lo teórico a lo práctico.
Parte práctica	Evidenciará el cumplimiento de todos los objetivos mediante código ejecutado.

CAPÍTULO III. MARCO TEÓRICO

3.1. Concepto de Python

Python es un lenguaje de programación de alto nivel, interpretado, de tipado dinámico y de propósito general, creado por Guido van Rossum y publicado por primera vez en 1991. Se caracteriza por su sintaxis legible que prioriza la claridad del código, lo que lo convierte en uno de los lenguajes más utilizados tanto en educación como en entornos profesionales. En la actualidad, Python lidera índices de popularidad como el TIOBE Index y PyPL, y es la primera opción en campos como la ciencia de datos, la inteligencia artificial, la automatización y la ciberseguridad.

En el ámbito de la seguridad informática, Python permite desarrollar herramientas de análisis de malware, scripts de automatización de auditorías, sistemas de monitoreo de red y motores de cifrado, gracias a su extensa biblioteca estándar y al ecosistema de librerías de terceros disponibles en PyPI (Python Package Index).

3.2. Concepto de librería en programación

Una librería es un conjunto de módulos, clases, funciones y recursos precompilados que permiten resolver tareas específicas dentro de un programa sin necesidad de implementar cada funcionalidad desde cero. Las librerías encapsulan lógica compleja y son distribuidas de forma que cualquier desarrollador pueda integrarlas mediante una simple instrucción de importación.

Ejemplo de importación de una librería en Python:

```
import hashlib
# La librería hashlib permite calcular funciones hash criptográficas
# como SHA-256 o MD5.
```

3.3. Clasificación de librerías de Python

Clasificación	Descripción	Ejemplos
Analítica de datos	Permiten limpiar, transformar, analizar y visualizar información estructurada.	Pandas, NumPy, Matplotlib, Seaborn
Computación gráfica	Permiten trabajar con imágenes, videojuegos, gráficos 2D/3D o visualizaciones.	OpenCV, Pillow, Pygame, PyOpenGL
Inteligencia artificial	Permiten crear modelos que aprenden de datos y realizan predicciones.	Scikit-learn, TensorFlow, PyTorch, Keras
Procesamiento de lenguaje natural	Permiten analizar textos, palabras, opiniones y lenguaje humano.	NLTK, spaCy, Transformers

Clasificación	Descripción	Ejemplos
Ciberseguridad defensiva	Permiten cifrar, verificar integridad, analizar archivos o protocolos de red.	hashlib, yara-python, impacket, Cryptography, Scapy

3.4. Importancia de las librerías en problemas reales

En el contexto de la ciberseguridad, las librerías de Python tienen un impacto directo en la capacidad de respuesta ante amenazas. hashlib permite a una organización verificar si un archivo descargado ha sido alterado comparando su hash con el valor original publicado por el proveedor. yara-python permite a un analista de SOC (Security Operations Center) escanear miles de archivos en segundos para detectar firmas de familias de malware conocidas. impacket, por su parte, es utilizada en auditorías de seguridad para simular ataques sobre protocolos de red y detectar vulnerabilidades antes de que actores maliciosos las exploten.

CAPÍTULO IV. DESARROLLO DE LAS LIBRERÍAS SELECCIONADAS

4.1. LIBRERÍA N.º 1: hashlib

4.1.1. Nombre de la librería Nombre oficial: hashlib. Se importa directamente bajo el mismo nombre: `import hashlib`. Es parte de la biblioteca estándar de Python, por lo que no requiere instalación adicional.

4.1.2. Definición de la librería hashlib es una librería estándar de Python que proporciona interfaces para los algoritmos de hash criptográficos más utilizados, incluyendo MD5, SHA-1, SHA-224, SHA-256, SHA-384, SHA-512 y BLAKE2. Una función hash transforma cualquier dato de entrada en una cadena de longitud fija llamada “digest” o resumen, de manera que un cambio mínimo en la entrada produce un resumen completamente diferente. Esta propiedad, conocida como efecto avalancha, es fundamental para verificar la integridad de archivos y contraseñas (Python Software Foundation, 2024).

4.1.3. Características principales

- Incluye algoritmos como MD5, SHA-1, SHA-256, SHA-512 y BLAKE2 sin instalación adicional.
- Permite calcular hashes de cadenas de texto, archivos y flujos de datos de forma eficiente.
- Es compatible con todas las plataformas donde corre Python (Windows, Linux, macOS).
- Nivel de dificultad: básico. Su interfaz es simple e intuitiva para principiantes.
- Se integra con otras librerías de ciberseguridad como hmac y secrets.

4.1.4. Instalación e importación hashlib es parte de la biblioteca estándar de Python. No requiere instalación con pip. Está disponible de forma inmediata en Google Colab, Kaggle y cualquier entorno Python.

```
# No se necesita instalación
import hashlib
```

4.1.5. Funciones o métodos principales

Función / Método	¿Para qué sirve?	Ejemplo breve
<code>hashlib.sha256()</code>	Crea un objeto hash SHA-256.	<code>h = hashlib.sha256()</code>

Función / Método	¿Para qué sirve?	Ejemplo breve
<code>hashlib.md5()</code>	Crea un objeto hash MD5. No recomendado para seguridad.	<code>h = hashlib.md5()</code>
<code>h.update(data)</code>	Alimenta datos al hash incrementalmente.	<code>h.update(b'mensaje')</code>
<code>h.hexdigest()</code>	Retorna el hash en hexadecimal.	<code>print(h.hexdigest())</code>
<code>h.digest()</code>	Retorna el hash como bytes.	<code>h.digest()</code>
<code>hashlib.algorithms_available</code>	Conjunto de algoritmos hash disponibles.	<code>hashlib.algorithms_available</code>
<code>hashlib.new('sha512', data)</code>	Crea un objeto hash por nombre de algoritmo.	<code>hashlib.new('sha512', b'dato')</code>

4.1.6. Ejemplo básico de uso El siguiente fragmento, extraído del módulo `comparar_hashes.py` del repositorio del proyecto, calcula simultáneamente los hashes MD5, SHA-1, SHA-256 y SHA-512 de dos archivos APK —uno original y otro envenenado— para detectar diferencias causadas por la inyección de un payload malicioso:

```
import hashlib, os, time

DIR = "/home/alim/exposicion final/APKs"
ARCHIVOS = ["CairosOriginal.apk", "CairosVenom.apk"]

def calcular_hashes(ruta):
    algoritmos = {
        "MD5": hashlib.md5(),
        "SHA-1": hashlib.sha1(),
        "SHA-256": hashlib.sha256(),
        "SHA-512": hashlib.sha512(),
    }
    with open(ruta, "rb") as f:
        for chunk in iter(lambda: f.read(4096), b''):
            for h in algoritmos.values():
                h.update(chunk)
    return {nombre: obj.hexdigest() for nombre, obj in algoritmos.items()}
```

Este diseño es eficiente porque lee el archivo en bloques de 4096 bytes, evitando cargar el archivo completo en memoria (los APK analizados superan los 52 MB). Además, calcular los cuatro hashes en una sola pasada reduce el tiempo de procesamiento.

Demostración del efecto avalancha: Una vez calculados los hashes SHA-256 de ambos APK, el script aplica una operación XOR bit a bit entre los dos digests para cuantificar el porcentaje de bits que cambiaron:

```
h1 = bytes.fromhex(original["SHA-256"])
h2 = bytes.fromhex(envenenado["SHA-256"])
bits_distintos = sum(bin(a ^ b).count('1') for a, b in zip(h1, h2))
total_bits = len(h1) * 8
porcentaje = (bits_distintos / total_bits) * 100
```

El resultado obtenido fue de 130 bits diferentes de 256 (50.8%), confirmando el efecto avalancha: la inyección del payload de Metasploit alteró aproximadamente la mitad de los bits del hash, haciendo imposible predecir el nuevo valor a partir del original.

4.1.7. Aplicación en contexto real En el marco de esta investigación se aplicó hashlib al análisis forense de dos aplicaciones Android reales: **CairosOriginal.apk** (52,400,958 bytes) y **CairosVenom.apk** (52,431,637 bytes). El segundo archivo fue generado mediante MsfVenom, inyectando un payload meterpreter_reverse_tcp en el APK original. El script **comparar_hashes.py** calculó los cuatro hashes de cada archivo, obteniendo valores completamente diferentes en todos los algoritmos (MD5, SHA-1, SHA-256 y SHA-512), lo que demuestra que la inyección del payload —que solo agrega 30,679 bytes al archivo— produjo un cambio radical en los resúmenes criptográficos. Este mismo principio es utilizado por plataformas como VirusTotal para detectar archivos modificados maliciosamente.

4.1.8. Ventajas y limitaciones

Ventajas	Limitaciones
Incluida en Python: no requiere instalación.	MD5 y SHA-1 son considerados inseguros para uso criptográfico crítico.
Interfaz simple y bien documentada.	No cifra datos: solo genera un resumen unidireccional (no es reversible).
Compatible con múltiples algoritmos de hash.	Para operaciones de contraseñas, se recomienda usar bcrypt o argon2 en lugar de hashlib.
Eficiente para procesar archivos de gran tamaño mediante bloques.	No gestiona claves ni certificados digitales por sí sola.

4.2. LIBRERÍA N.º 2: yara-python

4.2.1. Nombre de la librería Nombre oficial: yara-python. Se instala con `pip install yara-python` y se importa como: `import yara`. Es la interfaz Python de YARA (Yet Another Ridiculous Acronym), la herramienta de detección de malware más utilizada en entornos de análisis forense y respuesta a incidentes.

4.2.2. Definición de la librería yara-python es el módulo de Python que expone la API de YARA, un motor de detección de patrones diseñado para identificar y clasificar muestras de malware a partir de reglas definidas por el analista. YARA permite describir características de familias de malware mediante reglas que combinan cadenas de texto, patrones hexadecimales y condiciones lógicas. Fue creado originalmente por Víctor Manuel Álvarez y es mantenido actualmente por VirusTotal (Google) (VirusTotal, 2024).

4.2.3. Características principales

- Permite escribir reglas de detección basadas en patrones de texto, bytes hexadecimales y expresiones regulares.
- Analiza archivos, directorios y procesos en memoria en busca de coincidencias con las reglas definidas.
- Nivel de dificultad: intermedio. Requiere aprender la sintaxis de reglas YARA.
- Compatible con Python 3 y se integra con herramientas como Volatility y frameworks de análisis de malware.
- Es la librería base de plataformas profesionales como VirusTotal, Cuckoo Sandbox y MISP.

```
# Instalación
pip install yara-python

# Importación
import yara
```

4.2.4. Instalación e importación

4.2.5. Funciones o métodos principales

Función / Método	¿Para qué sirve?	Ejemplo breve
<code>yara.compile(source=...)</code>	Compila una regla YARA desde una cadena.	<code>r = yara.compile(source=s)</code>
<code>yara.compile(filepath=...)</code>	Compila reglas desde un archivo .yar.	<code>r = yara.compile(filepath='r.yar')</code>
<code>regla.match(filepath=...)</code>	Escanea un archivo y retorna coincidencias.	<code>matches = r.match(filepath='f.exe')</code>
<code>regla.match(data=...)</code>	Escanea datos en memoria (bytes).	<code>r.match(data=contenido)</code>
<code>matches[i].rule</code>	Nombre de la regla que coincidió.	<code>print(matches[0].rule)</code>
<code>matches[i].strings</code>	Cadenas que activaron la regla.	<code>print(matches[0].strings)</code>

4.2.6. Ejemplo básico de uso El siguiente fragmento, tomado de `detector_yara.py`, define dos reglas YARA para detectar payloads de Metasploit dentro del archivo `classes.dex` de un APK. Las reglas se compilan y posteriormente se aplican al DEX extraído del paquete:

```
import yara

REGLAS_DEX = yara.compile(source='''
    rule dex_metasploit_payload {
        meta:
            description = "Detecta payload de Metasploit en DEX"
            severity = "critical"
        strings:
            $a = "Ljava/lang/reflect/Method;"
            $b = "Ljava/net/URLConnection;"
            $c = "addRequestProperty"
            $d = ";>startService(Landroid/content/Context;)V"
            $e = ";>openConnection()Ljava/net/URLConnection;"
        condition:
            uint32(0) == 0x0A786564 and
            4 of them
    }
''')

# Extraer DEX del APK y escanear con YARA
with zipfile.ZipFile(ruta_apk, 'r') as z:
    for name in z.namelist():
        if name.endswith('.dex'):
            dex_data = z.read(name)
            break

matches = REGLAS_DEX.match(data=dex_data)
if matches:
    print('ALERTA: Payload detectado')
```

La regla `dex_metasploit_payload` busca cinco indicadores característicos de un RAT Android: uso de reflexión (`java/lang/reflect/Method`), conexiones de red (`java/net/URLConnection`), envío de cabeceras HTTP (`addRequestProperty`), inicio de servicios remotos y apertura de

conexiones URL. La condición exige que al menos cuatro de estos cinco patrones estén presentes, y además verifica que el archivo comience con el magic number de un DEX válido (0x0A786564).

Análisis combinado con Androguard: Además del escaneo con YARA, el detector implementa heurísticas complementarias: cuenta los permisos peligrosos declarados en el `AndroidManifest.xml` (24 en el APK envenenado frente a 2 en el original), identifica subpaquetes inyectados de nombre aleatorio corto (como `com.example.cairos.dfwto`), y compara el tamaño del DEX (993 KB vs 812 KB). Estos indicadores se combinan en una puntuación de riesgo que determina el veredicto final.

4.2.7. Aplicación en contexto real El detector `detector_yara.py` desarrollado en este proyecto analizó tres APK del repositorio: `CairosOriginal.apk`, `CairosVenom.apk` y `payload.apk`. El sistema identificó correctamente los dos APK envenenados mediante la combinación de reglas YARA sobre el DEX y heurísticas sobre el manifiesto Android. El APK original fue clasificado como LIMPIO (2 permisos, sin subpaquetes), mientras que el APK envenenado fue clasificado como ENVENENADO (24 permisos, 14 de ellos peligrosos, un subpaquete inyectado con nombre aleatorio de 5 caracteres). Este enfoque multicapa —YARA para firmas binarias y Androguard para análisis estructural— es el mismo que emplean las soluciones antimalware comerciales para Android.

4.2.8. Ventajas y limitaciones

Ventajas	Limitaciones
Es el estándar industrial para detección de malware basada en reglas.	Requiere aprender la sintaxis YARA para escribir reglas efectivas.
Altamente expresiva: combina strings, bytes, regex y lógica booleana.	Las reglas mal escritas pueden generar falsos positivos o negativos.
Compatible con miles de reglas públicas disponibles en repositorios como GitHub.	No detecta malware polimórfico avanzado que modifica sus patrones constantemente.
Permite escanear archivos, procesos en memoria y flujos de datos.	Requiere actualización constante de las reglas ante nuevas amenazas.

4.3. LIBRERÍA N.º 3: `impacket`

4.3.1. Nombre de la librería Nombre oficial: `impacket`. Se instala con `pip install impacket` y se accede a sus módulos mediante importaciones específicas según el protocolo: `from impacket.smbconnection import SMBConnection`, `from impacket import smb`, `ntlm`, entre otros.

4.3.2. Definición de la librería `impacket` es una colección de clases Python para trabajar con protocolos de red de bajo nivel, especialmente los utilizados en entornos Windows: SMB, MSRPC, NTLM, Kerberos, LDAP, MSSQL, WMI y DCOM. Fue desarrollada originalmente por SecureAuth Corporation y actualmente es mantenida por la comunidad a través de Fortra. Es ampliamente utilizada en auditorías de seguridad, análisis forense de red y laboratorios de seguridad ofensiva y defensiva (Fortra, 2024).

IMPORTANTE: `impacket` debe utilizarse exclusivamente en entornos propios, laboratorios controlados o con autorización escrita del propietario del sistema. Su uso no autorizado en sistemas de terceros constituye un delito informático.

4.3.3. Características principales

- Implementa protocolos como SMB1/2/3, Kerberos, NTLM, LDAP, MSSQL, WMI y DCOM en Python puro.

- Permite construir, enviar y analizar paquetes de red a nivel de protocolo.
- Nivel de dificultad: avanzado. Requiere conocimiento de protocolos de red Windows.
- Es la base de herramientas de seguridad reconocidas como BloodHound, crackmapexec y secretsdump.
- Incluye más de 20 scripts de ejemplo listos para usar en auditorías de seguridad.

```
# Instalación
pip install impacket

# Importaciones según el módulo necesario
from impacket.smbconnection import SMBConnection
from impacket import smb, ntlm
from impacket.krb5 import kerberosv5
```

4.3.4. Instalación e importación

4.3.5. Funciones o métodos principales

Módulo / Función	¿Para qué sirve?	Ejemplo breve
<code>SMBConnection()</code>	Establece conexión SMB a un servidor.	<code>conn = SMBConnection(ip, ip)</code>
<code>conn.login(user, pass)</code>	Autentica la conexión con credenciales.	<code>conn.login('user', 'pass')</code>
<code>conn.listShares()</code>	Lista recursos compartidos del servidor.	<code>shares = conn.listShares()</code>
<code>ntlm.getNTLMSSPType1()</code>	Construye mensaje NTLM Tipo 1.	<code>msg = getNTLMSSPType1()</code>
<code>smb.SMB.get_server_name()</code>	Obtiene el nombre del servidor SMB.	<code>server = get_server_name()</code>
<code>kerberosv5.getKerberosTGT()</code>	Solicita un TGT de Kerberos.	<code>tgt = getKerberosTGT(...)</code>

4.3.6. Limitaciones para la documentación práctica Durante el desarrollo de esta monografía se intentó generar una práctica funcional completa con impacket, similar a la realizada con hashlib y yara-python. Sin embargo, se encontraron las siguientes barreras técnicas y legales que imposibilitaron la ejecución de pruebas reales:

1. **Dependencia de un dominio Active Directory real:** Los módulos Kerberos de impacket requieren un Key Distribution Center (KDC) funcional para solicitar y validar Ticket Granting Tickets (TGT). Sin un controlador de dominio Windows Server configurado con usuarios, grupos y políticas de autenticación, las funciones `getKerberosTGT()` y `getKerberosTGS()` retornan errores de conexión irreversibles. Montar un laboratorio completo con Windows Server, roles de dominio y cuentas de servicio excede el alcance de este trabajo monográfico.
2. **Necesidad de credenciales privilegiadas:** Incluso en un entorno de laboratorio, los scripts de impacket como `secretsdump.py`, `GetNPUsers.py` y `wmiexec.py` requieren autenticación con cuentas de dominio que posean privilegios específicos (e.g., permisos de replicación de directorio para DCSync). La generación de estas credenciales implica la administración completa de un dominio, actividad que requiere autorización institucional formal.
3. **Complejidad del código base:** impacket implementa más de 20 módulos que abarcan protocolos completos a nivel de paquete. Cada módulo encapsula estructuras ASN.1, mecanismos de negociación GSS-API, cifrado RC4-HMAC y AES para Kerberos, y parsing de

PAC (Privilege Attribute Certificate). Documentar un flujo completo requeriría desensamblar y explicar miles de líneas de código que operan a niveles del stack OSI que exceden los objetivos de esta investigación.

4. **Implicaciones éticas y legales:** La ejecución de herramientas de impacket contra sistemas que no son de propiedad del equipo investigador constituiría un delito informático tipificado en la legislación peruana (Ley N° 30096, Ley de Delitos Informáticos). Por ello, la documentación se limita al análisis conceptual de la arquitectura de la librería y sus métodos principales, sin ejecución contra infraestructura real.

No obstante, se reconoce que impacket es una herramienta de valor incalculable en auditorías de seguridad profesionales. Su inclusión en esta monografía responde al objetivo de documentar su existencia, arquitectura y aplicaciones teóricas, dejando la práctica operativa para entornos controlados con las autorizaciones correspondientes.

4.3.7. Aplicación en contexto real Un auditor de seguridad contratado por una empresa utiliza impacket en un entorno controlado para verificar si los servidores Windows de la organización tienen recursos SMB expuestos sin autenticación adecuada, o si el protocolo NTLM está configurado de forma vulnerable. Estas pruebas, realizadas con autorización formal, permiten identificar vectores de ataque antes de que actores externos los exploten, contribuyendo directamente a la postura de seguridad de la organización.

4.3.8. Ventajas y limitaciones

Ventajas	Limitaciones
Implementación Python pura de protocolos de red Windows sin dependencias externas.	Nivel avanzado: requiere sólidos conocimientos de protocolos de red.
Base de herramientas profesionales ampliamente utilizadas en la industria.	Su uso fuera de entornos autorizados puede constituir un delito.
Permite analizar y entender protocolos como SMB y Kerberos a nivel de paquete.	La documentación oficial es técnica y puede ser difícil para principiantes.
Actualización constante para cubrir nuevas versiones de protocolos.	Requiere un entorno de laboratorio (máquina virtual) para practicar de forma segura.

CAPÍTULO V. CUADRO COMPARATIVO DE LAS LIBRERÍAS

Criterio	hashlib	yara-python	impacket
Área de aplicación	Verificación de integridad criptográfica.	Detección y clasificación de malware.	Análisis y simulación de protocolos de red.
Uso principal	Calcular hashes MD5, SHA-256, SHA-512 de archivos y datos.	Escanear archivos y memoria con reglas de patrones YARA.	Implementar y analizar protocolos SMB, NTLM, Kerberos, LDAP.
Nivel de dificultad	Básico	Intermedio	Avanzado
Tipo de datos que maneja	Strings, bytes, archivos de cualquier tipo.	Archivos, procesos en memoria, flujos de bytes.	Paquetes de red, sesiones SMB, tickets Kerberos.
Funciones más importantes	<code>sha256()</code> , <code>md5()</code> , <code>update()</code> , <code>hexdigest()</code>	<code>compile()</code> , <code>match()</code> , <code>strings</code> , <code>meta</code>	<code>SMBConnection</code> , <code>login()</code> , <code>listShares()</code> , NTLM
Ventajas principales	Estándar, sin instalación, interfaz simple.	Estándar industrial en análisis de malware.	Implementa protocolos reales en Python puro.
Limitaciones	No cifra, solo genera resúmenes; MD5/SHA-1 inseguros.	Requiere aprender sintaxis YARA; no detecta malware polimórfico avanzado.	Nivel avanzado; uso solo en entornos autorizados.
Ejemplo de uso real	Verificar integridad de instaladores de software.	Escanear adjuntos de correo en busca de malware.	Auditar configuraciones SMB/Kerberos en red interna.

Criterio	hashlib	yara-python	impacket
Relación con la práctica	Cálculo y comparación de hashes de APKs de 52 MB; efecto avalancha con XOR bit a bit.	Detección de payload Metasploit en DEX + análisis de permisos y subpaquetes con Androguard.	Documentación teórica; no se ejecutó práctica por requerir dominio Active Directory y credenciales autorizadas.

Análisis interpretativo del cuadro comparativo

Al comparar las tres librerías, se observa una progresión natural en términos de complejidad y alcance dentro de la ciberseguridad defensiva. hashlib resultó la más accesible para principiantes por su interfaz directa y su inclusión en la biblioteca estándar, siendo ideal como punto de entrada al estudio de la criptografía aplicada. En la práctica realizada, procesó archivos de 52 MB en 0.37 segundos calculando cuatro hashes simultáneamente, y la comparación XOR bit a bit del SHA-256 cuantificó el efecto avalancha en 50.8%.

yara-python exige un nivel intermedio de conocimiento, especialmente en la sintaxis de reglas YARA, pero ofrece una capacidad de detección de amenazas que se acerca a los entornos profesionales reales. En este proyecto, se demostró que la combinación de reglas YARA sobre el DEX con heurísticas de Androguard (conteo de permisos, detección de subpaquetes inyectados) produce una clasificación precisa de APKs envenenados.

impacket es la herramienta de mayor complejidad técnica, orientada a profesionales con conocimientos consolidados de redes, protocolos Windows y administración de dominios Active Directory. Su documentación práctica no pudo ser completada porque los módulos Kerberos requieren un KDC funcional, los scripts de auditoría necesitan credenciales privilegiadas de dominio, y la ejecución sin autorización constituiría un delito informático. No obstante, se documentó su arquitectura, métodos principales y aplicaciones teóricas.

Las tres librerías son complementarias: hashlib garantiza que los archivos no han sido modificados, yara-python detecta si contienen código malicioso, e impacket permite analizar cómo se comunican los sistemas en red. Juntas, conforman una cadena de análisis defensivo coherente, aunque impacket requiere condiciones de laboratorio que exceden el alcance típico de un trabajo monográfico de pregrado.

CAPÍTULO VI. PARTE PRÁCTICA: ANÁLISIS FORENSE DE APKs

6.1. Plataforma utilizada

Se utilizó un entorno local Linux con Python 3.14, un entorno virtual (`venv`) con las dependencias yara-python y androguard, y las herramientas del sistema apktool, msfvenom y Metasploit Framework para la generación del APK envenenado. Para la versión de Google Colab, se encuentran disponibles dos notebooks: uno para hashlib y otro para yara-python (ver Anexo 4). Los APK utilizados y todo el código fuente están publicados en el repositorio GitHub del proyecto.

6.2. Descripción del problema práctico

El problema práctico consiste en determinar, mediante herramientas Python, si un archivo APK ha sido manipulado con un payload malicioso inyectado por MsfVenom (Metasploit). Se trabaja con tres archivos: `CairosOriginal.apk` (aplicación Flutter legítima, 52.4 MB), `CairosVenom.apk` (el mismo APK con un payload meterpreter_reverse_tcp inyectado, 52.4 MB) y `payload.apk` (APK standalone generado directamente por msfvenom, 10 KB). El análisis se realiza en dos fases: (1) comparación de hashes criptográficos para verificar la

integridad, y (2) detección automatizada mediante reglas YARA y heurísticas estructurales con Androguard.

6.3. Datos utilizados

- CairoOriginal.apk (52,400,958 bytes): APK legítimo original.
- CairoVenom.apk (52,431,637 bytes): APK original con payload Metasploit inyectado.
- payload.apk (10,246 bytes): APK standalone de msfvenom.

Todos los APK se encuentran en la carpeta **APKs/** del repositorio.

6.4. Código desarrollado

Fase 1: Comparación de hashes con hashlib

El script `comparar_hashes.py` calcula MD5, SHA-1, SHA-256 y SHA-512 de los dos APK principales y compara bit a bit los resúmenes:

```
import hashlib, os, time

DIR = "/home/alim/exposicion final/APKs"
ARCHIVOS = ["CairosOriginal.apk", "CairosVenom.apk"]

def calcular_hashes(ruta):
    algoritmos = {
        "MD5": hashlib.md5(),
        "SHA-1": hashlib.sha1(),
        "SHA-256": hashlib.sha256(),
        "SHA-512": hashlib.sha512(),
    }
    with open(ruta, "rb") as f:
        for chunk in iter(lambda: f.read(4096), b''):
            for h in algoritmos.values():
                h.update(chunk)
    return {nombre: obj.hexdigest() for nombre, obj in algoritmos.items()}
```

Fase 2: Detección con YARA + Androguard

El script `detector_yara.py` implementa un sistema de puntuación que combina reglas YARA sobre el DEX con heurísticas del manifiesto Android:

```
from androguard.core.apk import APK

PERMISOS_PELIGROSOS = [
    "android.permission.READ_SMS",
    "android.permission.SEND_SMS",
    "android.permission.CAMERA",
    "android.permission.RECORD_AUDIO",
    "android.permission.READ_CALL_LOG",
]

apk = APK(ruta)
permisos = apk.get_permissions()
peligrosos = [p for p in permisos if p in PERMISOS_PELIGROSOS]
```

```
# Detectar subpaquete inyectado (nombre aleatorio corto)
main_package = apk.get_package()
subpaquetes = [p for p in paquetes
                if p.startswith(main_package) and p != main_package]
```

6.5. Explicación del código

La Fase 1 utiliza lectura por bloques de 4 KB para procesar archivos de más de 50 MB sin saturar la memoria RAM. Calcula cuatro algoritmos en una sola pasada, optimizando el tiempo de ejecución (0.37 segundos para 52 MB). La comparación XOR bit a bit del SHA-256 demuestra cuantitativamente el efecto avalancha.

La Fase 2 emplea un enfoque de detección por capas: (a) reglas YARA que buscan firmas de código malicioso en el DEX (reflexión, conexiones de red, inicio de servicios); (b) análisis del AndroidManifest.xml mediante Androguard para contar permisos peligrosos declarados; (c) detección de subpaquetes inyectados con nombres aleatorios de 5 caracteres generados por Msf-Venom; (d) comparación del tamaño del DEX (el payload agrega 181,696 bytes). Cada indicador suma puntos a una puntuación de riesgo; si el puntaje supera 5, el APK se clasifica como ENVENENADO.

6.6. Resultados obtenidos

Indicador	CairosOriginal	CairosVenom	payload.apk
Tamaño	52,400,958 B	52,431,637 B	10,246 B
Permisos totales	2	24	23
Permisos peligrosos	0	14	14
Subpaquetes inyectados	0	1 (dfwto)	0
Tamaño DEX	812 KB	993 KB	20 KB
SHA-256 (primeros 16)	6752e6831ed260a7	fe1452b4900c9214	—
Veredicto	LIMPIO	ENVENENADO	ENVENENADO
Bits SHA-256 distintos	—	130/256 (50.8%)	—

6.7. Evidencias de ejecución

- Capturas de pantalla de la terminal con la ejecución de `comparar_hashes.py` mostrando los cuatro hashes de cada APK.
- Salida de `detector_yara.py` clasificando los tres APK con su puntuación de riesgo e indicadores.
- Notebook de Google Colab con la demostración de hashlib:
<https://colab.research.google.com/drive/1oXyDNrLdfR4-8oltR1WURTp8IMK65TTF>
- Notebook de Google Colab con la demostración de yara-python:
https://colab.research.google.com/drive/173T7NiHw52sU_ySdK0RQoJzm5KdplAmp
- Repositorio GitHub completo:
<https://github.com/AlvaroHuacacSoto/hashlib-Jara>

CAPÍTULO VII. CASO PRÁCTICO: DETECCIÓN DE APK ENVENENADO

7.1. Descripción del caso

Un desarrollador independiente descubre que una versión modificada de su aplicación Android **Cairos.apk** está circulando en foros de terceros. La versión legítima es una app Flutter de 52 MB que solo requiere permisos de INTERNET y un permiso interno de broadcast. La versión

sospechosa tiene el mismo tamaño aproximado, pero al instalarse solicita acceso a SMS, cámara, micrófono, contactos y registros de llamadas. El desarrollador necesita determinar técnicamente si el APK fue manipulado y cómo.

7.2. Problema identificado

- El APK circulante solicita 24 permisos (14 peligrosos), frente a los 2 permisos de la versión legítima.
- El hash criptográfico del archivo no coincide con el original publicado por el desarrollador.
- Se sospecha la inyección de un RAT (Remote Access Trojan) mediante herramientas de código abierto como Metasploit/MsfVenom.
- Es necesario un método automatizado y reproducible para detectar la manipulación sin depender de inspección manual.

7.3. Solución propuesta con Python

Se desarrollaron dos scripts Python que abordan el problema desde dos ángulos complementarios:

- **comparar_hashes.py:** calcula y compara los hashes MD5, SHA-1, SHA-256 y SHA-512 de ambos APK. La diferencia en los cuatro algoritmos confirma que el archivo fue alterado. El análisis XOR del SHA-256 revela que el 50.8% de los bits cambiaron, lo cual es consistente con la inyección de un payload de aproximadamente 30 KB dentro de un archivo de 52 MB.
- **detector_yara.py:** implementa un motor de detección que combina (a) reglas YARA compiladas para buscar patrones de RAT en el DEX (reflexión, conexiones URL, inicio de servicios remotos), (b) análisis del manifiesto mediante Androguard para contar permisos peligrosos y detectar subpaquetes inyectados con nombres aleatorios, y (c) comparación del tamaño del DEX. El sistema asigna una puntuación de riesgo y emite un veredicto (LIMPIO / SOSPECHOSO / ENVENENADO).

7.4. Beneficios de la solución

- El desarrollador puede verificar la integridad de su APK en segundos comparando el hash SHA-256 con el valor de referencia.
- El detector YARA + Androguard identifica automáticamente APKs envenenados sin necesidad de ingeniería inversa manual.
- El enfoque es reproducible: cualquier analista puede clonar el repositorio, instalar las dependencias y ejecutar los scripts sobre nuevos APKs sospechosos.
- La metodología es aplicable a la detección de malware Android en tiendas de aplicaciones alternativas, repositorios de APK y campañas de phishing móvil.

Elemento del caso	Descripción
Contexto	Desarrollador de app Flutter descubre versión alterada de su APK en distribución no autorizada.
Actor principal	Desarrollador / Analista de seguridad móvil.
Problema	APK alterado con 24 permisos (14 peligrosos) vs 2 permisos originales; hashes no coinciden.
Datos o archivos	CairosOriginal.apk (52.4 MB), CairosVenom.apk (52.4 MB), payload.apk (10 KB).
Librerías	hashlib (verificación de integridad), yara-python + androguard (detección automatizada), impacket (documentación teórica).
Herramienta de ataque identificada	MsfVenom (Metasploit Framework) con payload android/meterpreter/reverse_tcp.

Elemento del caso	Descripción
Resultado obtenido	Detección exitosa: original clasificado LIMPIO, envenenado clasificado ENVENENADO con 14 indicadores de peligro.
Repositorio	https://github.com/AlvaroHuacacSoto/hashlib-Jara

CONCLUSIONES

1. Las librerías hashlib, yara-python e impacket demuestran que Python es una plataforma robusta para implementar soluciones de ciberseguridad defensiva. Cada librería aborda una dimensión diferente del problema de seguridad: integridad (hashlib), detección (yara-python) y análisis de protocolos (impacket). El proyecto práctico desarrollado —análisis forense de APKs envenenados con payload de Metasploit— demostró la efectividad de las dos primeras en un escenario real, mientras que impacket fue documentada teóricamente por las barreras de autenticación que requiere.
2. El análisis de hashes con `comparar_hashes.py` confirmó el efecto avalancha: los cuatro algoritmos (MD5, SHA-1, SHA-256, SHA-512) produjeron valores completamente diferentes entre el APK original y el envenenado, con 130 de 256 bits (50.8%) distintos en SHA-256. Esto valida el uso de hashes criptográficos como primera línea de defensa contra manipulaciones de software.
3. El detector `detector_yara.py` clasificó correctamente los tres APK: `CairosOriginal.apk` como LIMPIO (2 permisos), `CairosVenom.apk` como ENVENENADO (24 permisos, subpaquete inyectado `dfwto`) y `payload.apk` como ENVENENADO. La combinación de reglas YARA con heurísticas de Androguard demostró ser más efectiva que cualquiera de las técnicas por separado.
4. El caso práctico evidenció que herramientas de código abierto como MsfVenom pueden ser utilizadas para generar APKs maliciosos indetectables a simple vista (mismo ícono, nombre y tamaño similar), pero las librerías Python de ciberseguridad permiten identificarlos mediante análisis automatizado de su estructura interna.

RECOMENDACIONES

- Clonar el repositorio <https://github.com/AlvaroHuacacSoto/hashlib-Jara> que contiene todos los scripts, APKs de prueba y este documento para replicar los experimentos.
- Revisar los notebooks de Google Colab publicados como complemento interactivo a esta monografía.
- Al usar hashlib para verificación de integridad, preferir SHA-256 o SHA-512 y evitar MD5/SHA-1 por sus vulnerabilidades de colisión conocidas.
- Combinar reglas YARA con análisis estructural del manifiesto Android para reducir falsos negativos en la detección de malware móvil.
- Para trabajar con impacket, montar un laboratorio controlado con Windows Server y Active Directory, y obtener autorización formal antes de ejecutar cualquier script contra infraestructura que no sea propia.
- Mantener actualizadas las reglas YARA incorporando firmas de repositorios comunitarios como YARA-Rules de GitHub y las colecciones de VirusTotal.

REFERENCIAS BIBLIOGRÁFICAS

PYTHON SOFTWARE FOUNDATION, 2024. *hashlib — Secure hashes and message digests* [en línea]. Python 3.12 Documentation. Disponible en: <https://docs.python.org/3/library/hashlib.html> [Consultado: 29 de junio de 2026].

VIRUSTOTAL / GOOGLE, 2024. *YARA documentation* [en línea]. GitHub Pages. Disponible en: <https://virustotal.github.io/yara/> [Consultado: 29 de junio de 2026].

FORTRA, 2024. *impacket: A collection of Python classes for working with network protocols* [en línea]. GitHub. Disponible en: <https://github.com/fortra/impacket> [Consultado: 29 de junio de 2026].

VAN ROSSUM, G. y DRAKE, F. L., 2009. *Python 3 Reference Manual* [en línea]. Python Software Foundation. Disponible en: <https://docs.python.org/3/reference/> [Consultado: 29 de junio de 2026].

ÁLVAREZ, V. M., 2014. *YARA: The pattern matching swiss knife for malware researchers* [en línea]. VirusTotal Blog. Disponible en: <https://www.virustotal.com/gui/blog> [Consultado: 29 de junio de 2026].

SECUREAUTH CORPORATION, 2023. *Impacket scripts and examples for network protocol analysis* [en línea]. GitHub. Disponible en: <https://github.com/fortra/impacket/tree/master/examples> [Consultado: 29 de junio de 2026].

HUACAC SOTO, A., 2026. *hashlib-Jara: Detección de APKs envenenados con Python* [en línea]. GitHub. Disponible en: <https://github.com/AlvaroHuacacSoto/hashlib-Jara> [Consultado: 29 de junio de 2026].

RAPID7, 2024. *Metasploit Framework Documentation* [en línea]. Disponible en: <https://docs.metasploit.com/> [Consultado: 29 de junio de 2026].

ANDROGUARD, 2024. *Androguard: Reverse engineering tool for Android applications* [en línea]. GitHub. Disponible en: <https://github.com/androguard/androguard> [Consultado: 29 de junio de 2026].

ANEXOS

Anexo 1: Código fuente completo

El código fuente completo de los scripts `comparar_hashes.py` y `detector_yara.py` se encuentra en el repositorio GitHub: <https://github.com/AlvaroHuacacSoto/hashlib-Jara>

Anexo 2: Resultados de ejecución

`comparar_hashes.py`

`CairosOriginal.apk`

```
MD5      : 3392a4d54f89fc7fe9c6618ff418bd75
SHA-1    : 1e4dc3d80c74e4a5f2aafb0cbd2de80119180708
SHA-256  : 6752e6831ed260a7a53d79c4607084b392f31c6b0d527a446faacdbb283c2b7a
SHA-512  : 201a5bacda49459a19fc3a66356ab3707d30d12ad1c4c7969c9b4415362fccd5...
```

`CairosVenom.apk`

```
MD5      : 7a35a20a63573b0225c49be3a3f55295
SHA-1    : cfb3bba3068982c74c180b3a980166a072938b4f
SHA-256  : fe1452b4900c9214af83e81b7bada97a598a900be32fd9962f3a59c26c69abe4
SHA-512  : 3708cf576beda849566697649088ecee4515d36b572239261bc447a99164ae25...
```

RESULTADO: Todos los hashes DISTINTOS.

Efecto avalancha SHA-256: 130/256 bits (50.8%)

`detector_yara.py`

CairosOriginal.apk: LIMPIO
CairosVenom.apk: ENVENENADO
payload.apk: ENVENENADO

APKs ENVENENADOS (2):

- CairosVenom.apk (14 permisos peligrosos, subpaquete dfwto)
- payload.apk (14 permisos peligrosos)

APKs LIMPIOS (1):

- CairosOriginal.apk (2 permisos)

Anexo 3: Archivos de prueba utilizados

- APKs/CairosOriginal.apk — APK legítimo (52,400,958 bytes)
- APKs/CairosVenom.apk — APK con payload Metasploit (52,431,637 bytes)
- APKs/payload.apk — APK standalone de MsfVenom (10,246 bytes)

Anexo 4: Enlaces a notebooks de Google Colab

- Notebook hashlib:
<https://colab.research.google.com/drive/1oXyDNrLdfR4-8oltR1WURTp8IMK65TTF>
- Notebook yara-python:
https://colab.research.google.com/drive/173T7NiHw52sU_ySdK0RQoJzm5KdplAmp

Anexo 5: Repositorio del proyecto

<https://github.com/AlvaroHuacacSoto/hashlib-Jara>

Contiene: scripts Python, APKs de prueba, documento PDF y entorno virtual con dependencias.

Anexo 6: Regla YARA utilizada

```
rule dex_metasploit_payload {
  meta:
    description = "Detecta payload de Metasploit en DEX"
    author = "Lab Seguridad"
    severity = "critical"
  strings:
    $a = "Ljava/lang/reflect/Method;"
    $b = "Ljava/net/URLConnection;"
    $c = "addRequestProperty"
    $d = ";>startService(Landroid/content/Context;)V"
    $e = ";>openConnection()Ljava/net/URLConnection;"
  condition:
    uint32(0) == 0x0A786564 and
    4 of them
}

rule dex_payload_backdoor {
  meta:
    description = "Detecta estructura tipica de RAT/backdoor Android"
    severity = "high"
  strings:
    $ref = "Ljava/lang/reflect/Method;"
```

```
    $start = ";>startService(Landroid/content/Context;)V"
condition:
    uint32(0) == 0x0A786564 and
    all of them
}
```